hermes-mini

A Serverless Persian Task Assistant on Cloudflare — Telegram & Bale, Multi-User

Author: Mojtaba (mojtaba.err@gmail.com) **Date:** Aban 1405 / November 2026 **Live URL:** https://hermes-mini.mojtaba-dev.workers.dev **Stack:** Workers + D1 + Gemini

1 · Overview

hermes-mini is a task assistant you talk to in plain, conversational Persian: «میان‌ترم درس رایانش ابری دوشنبه هفته دیگه ساعت ۱۰ دارم» becomes a stored task with a reminder two days earlier. There are **no slash commands** — understanding the message and deciding the action is entirely the language model's job (R1, R2). The whole system runs on the Cloudflare **free plan**: a single Worker file, a D1 SQLite database, a cron trigger, and the Gemini API behind a round-robin of ten API keys. It serves **multiple whitelisted users** and supports the Iranian messenger **Bale** alongside Telegram. State (tasks, chat memory, key-rotation cursor, user whitelist) lives entirely in D1; the Worker itself is stateless.

2 · Architecture

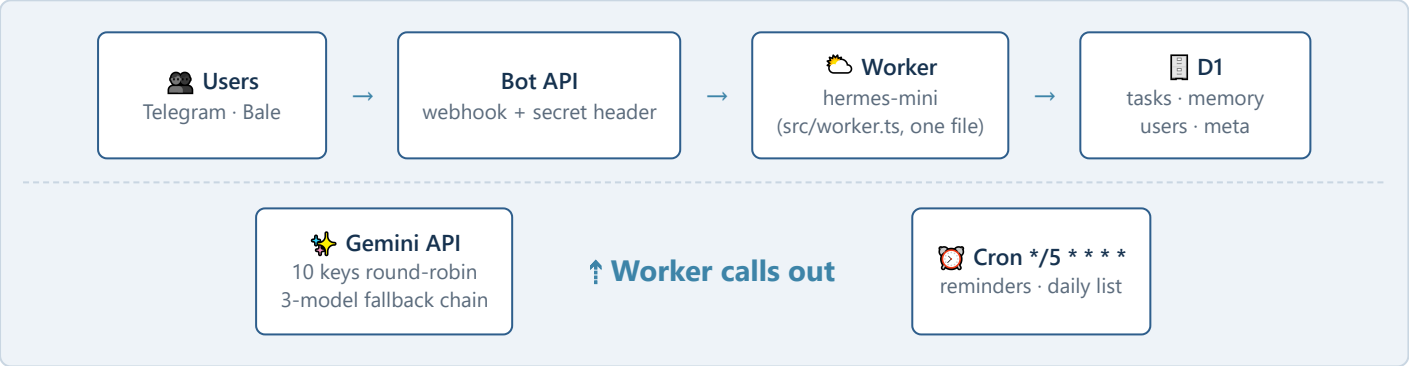


Figure 1 — Event flow. The Worker answers the webhook, decides with Gemini, acts on D1, and replies on the user's own messenger. Cron drives reminders and the morning list.

3 · Design Decisions

Decision	Justification
Webhook, not polling	The Worker runs only when an event arrives — no idle cost, no keep-alive process. Polling would burn requests around the clock and needs a persistent consumer, which FaaS does not offer.
FaaS for this workload	Traffic is a few dozen messages a day. A VM would be ~100% idle; Workers bill per request with a 100k/day free allowance — effectively $0.
D1, not KV	Tasks need real queries: due-time windows for reminders, fuzzy title matching, per-user streak aggregation, id-based updates. KV is a whole-value lookup with no queryability; D1 is SQLite with indexes and transactions.
State lives in D1	Tasks, rolling chat memory, whitelist, and the Gemini key-rotation cursor are all rows. Any isolate can serve any request; restarts lose nothing.
Tehran time	The runtime has no timezone. Wall-clock strings (due_date, due_time) are Tehran-local; comparisons use epoch-ms. Tehran = UTC+03:30 *fixed* (Iran abolished DST in 2022), so `tehranNow()` shifts `Date.now()` by 210 minutes and reads UTC fields. Display uses Jalali dates via a bundled, unit-tested conversion (no library).
One worker file	All logic fits in a single `src/worker.ts` (~950 lines) — the hard "no frameworks" constraint, kept deliberately.

4 · The Intelligence Layer

Every message is sent to Gemini together with three pieces of ground truth: the **current Tehran datetime** (Gregorian + Jalali + weekday, so relative phrases like «دوشنبه هفته دیگه» resolve deterministically), the **numbered list of open tasks**, and the **last 12 chat turns** (R4). The model must answer with a single line of JSON — enforced by `responseMimeType: application/json`:

```
{"action":"add|done|undone|del|edit|search|stats|list|chat","task":…,"date":"YYYY-MM-DD","time":"HH:MM","remind_minutes":…,"category":…,"scope":"open|done","target_ids":[…],"target_query":…,"all":false,"reply":"…"}
```

Validation & safe fallback (R3)

- A hand-rolled validator checks the action set, date format & range (not in the past), reminder ≤ 30 days, `add` needs a title, `done/del` need a target, `edit` needs a target *and* a change.
- On invalid output the model is shown its own output plus the error and retries **once**.
- If it still fails → a Persian apology is sent and **nothing is written to D1**. Malformed output can never corrupt data.

Target resolution

- By id from the numbered list, or fuzzy Persian title match (ك→ک، ي→ی، Arabic→Persian digits, ZWNJ stripped).
- Ambiguous matches make the bot **ask which one** — never guess (graded robustness case).
- Ids are looked up across all statuses, so «شماره‌ی ۳ رو حذف کن» works even for done tasks.

Round-robin & model chain

- **10 keys** rotate on every call; the cursor persists in D1 and survives restarts.
- Failures are classified: dead key (400/401/403) → next key; overload (404/429/5xx) → **next model** in the chain: `flash-lite → 3.6-flash → flash-latest`.
- A **14 s hard deadline** plus 8 s per-call timeouts bound the whole chain — the bot always answers something.
- 3.x models run with `thinkingLevel: "low"`.

> **Key lesson:** Gemini 3.x models "think" by default (~400+ thought tokens per decision). Benchmarked `thinkingLevel: "low"` cut a typical decision from 11.2 s to 3.6 s with no contract-quality loss.

Live model benchmark (production prompt, 5 real cases)

Model	Success	Contract quality	Avg latency	Verdict
gemini-3.6-flash	3 / 5	3 / 3 of answered	5.6 s	Smart but flaky (503/timeout that day)
gemini-3.5-flash	1 / 5	–	–	Hard 429 rate-limiting — dropped
gemini-3.5-flash-lite	**5 / 5**	4 / 5	8.9 s → **3.6 s** (thinking-low)	**Chosen primary**
gemini-3.1-flash-lite	2 / 5	2 / 2	5.7 s	Overloaded — kept out

Table 1 — Benchmark run live against the real system prompt and validator (`tests/bench-models.mts`). The one "≈" case was relative-date reasoning, mitigated by prompting rules + the fallback chain.

5 · Reliability Engineering

- **Inline processing:** the webhook is answered 200 *after* the work. An early design did the work in `ctx.waitUntil` — when Google 503s responded slowly, the ~30 s waitUntil budget killed the task mid-flight and replies silently vanished. Inline + deadlines removed the failure mode entirely (every outcome answers 200, so Telegram never redelivers).
- **Cron (bonus):** `*/5 * * * *` fires reminders due within ≤5 min (joined with the users table so each reminder lands on the right messenger), and once a day at a fixed Tehran hour sends every user their grouped list plus a streak line.
- **Health & observability:** `GET /health` on the live URL; unsigned webhooks get **403** (R6); strangers are rejected **silently** (R5, now a D1 whitelist); observability is enabled and demoed with `wrangler tail`.
- **46 unit tests** cover the pure core: Jalali conversions (Nowruz + leap-year fixtures), the decision contract, key parsing, Persian normalization, list rendering — all passing.

6 · Features & the Action Contract

Action	Behavior	Persian example → result
add	Stores a task with due date/time, category tag, and an LLM-chosen reminder for important work (exams 2 days before, meetings 1 day).	«شیر بخر» → ثبت در لیست؛ «امتحان فردا هفته دیگه» → ثبت + یادآوری ۲ روز قبل
done	Ticks a task off (cancels its reminders).	«شیر رو خریدم» → تیک خورد ✅
undone	Re-opens a wrongly-ticked task.	«اشتباه تیک خورد» → برگشت به باز
del	Deletes by id/query; `all` empties open tasks; `scope:"done"` empties the archive.	«همه رو حذف کن» → پاک شدن همه‌ی کارهای باز
edit	Changes title/date/time/reminder/category. The stored reminder offset is re-derived automatically when the due time moves.	«جلسه رو ببر شنبه» → تاریخ جدید + یادآوری جابه‌جا
search	Searches open *and* archived tasks (title + category).	«اون آزمون کی بود؟» → نتایج با وضعیت ✅/▫
stats	Done-counts per Tehran day, current & best streak.	«استریکم چیه؟» → 🔥 ۵ روز پشت‌سرهم
list	Grouped list: overdue ⚠, today, tomorrow, this week (weekday), later (Jalali date), no-date.	«کارام چیه؟» → فهرست شماره‌دار با برچسب دسته
chat	Anything else — with 12-turn conversational memory and clarification questions on ambiguity.	«سلام» → پاسخ گفتگویی طبیعی

7 · Multi-User & Bale

- The whitelist is the D1 `users (platform, user_id)` table; tasks are scoped per platform so the same numeric id on two messengers never collides. Adding a user is one SQL insert (no redeploy).
- Reminders and the daily morning list are delivered on **each user's own messenger** (Telegram or Bale).
- **Bale** mirrors Telegram's Bot API: the worker serves it at `POST /bale`, its bot token/secret are separate secrets, and registration is `node scripts/set-webhook.mjs <url> --bale` (base `tapi.bale.ai`).

8 · Live Transcripts (raw, from the deployed bot)

E R: فردا جلسه میرم
hermes: جلسه رو برای فردا، چهارشنبه ۱۳ آبان ثبت کردم. خواستی بگو چند ساعت قبل هم یادت بندازم. ✅ «جلسه» ثبت شد — فردا ⏰ یک روز قبل یادت می‌اندازم.
E R: ساعت 9 صبح
hermes: جلسه‌ات رو برای فردا، چهارشنبه ۱۳ آبان ساعت ۹ صبح به‌روزرسانی کردم. ✅ «جلسه» ثبت شد — فردا ساعت ۰۹:۰۰
E R: 2 ساعت قبلش خبر بده
hermes: حله، یادآوری جلسه رو برای ۲ ساعت قبلش (ساعت ۷ صبح فردا) تنظیم کردم. ⏰ ۲ ساعت قبل یادت می‌اندازم.

E R: سه شنبه هفته دیگه امتحان دارم
hermes: امتحانت رو برای سه‌شنبه ثبت کردم. چون امتحان مهمه، دو روز قبلش یادت می‌ندازم؛ خواستی بگو تغییرش بدم. ✅ «امتحان» ثبت شد ⏰ ۲ روز قبل یادت می‌اندازم.
E R: فردا جلسه نمیرم
hermes: کدوم جلسه‌ات رو لغو کنم؟ چند تا جلسه داری؛ شماره‌ش رو بگو. (ambiguity → clarification, no guess)
E R: همه رو حذف کن
hermes: همه‌ی کارهات رو پاک کردم. اگر کار تازه‌ای داشتی، بهم بگو. 🗑

E R: آمارم رو نشون بده
hermes: 📊 آمار کارهایت: ✅ امروز: ۳ کار · 📆 ۷ روز اخیر: ۱۲ کار · 🔥 استریک فعلی: ۵ روز پشت‌سرهم · 🏆 رکورد: ۹ روز · 📦 آرشیو: ۴۸ کار (sample output)

Figure 2 — Transcript excerpts (Aban 1405). Note reminder negotiation by pure conversation, the ambiguity clarification, and Persian-only UX throughout.

9 · Cost Analysis at Two Scales

Metric (per day)	1 user	10,000 users	Free-tier limit
Worker requests (webhooks + cron)	~320	~260,000	100,000 (free)
D1 rows read	~2,000	~5,000,000	5,000,000 (free)
D1 rows written	~200	~300,000	100,000 (free)
Gemini calls (≈1.5k tokens each)	~25	~250,000	per-key quota ×10 keys
Cloudflare cost / month	**$0**	≈ $10 (Workers Paid $5 + D1 paid tier)	–
LLM cost / month (raw)	**$0** (free tier)	≈ $1,300–1,800	–
LLM cost / month (cached + batched)	**$0**	≈ $400–700 (≈ 4–7¢ / user)	–

Table 2 — Order-of-magnitude estimates, flash-class pricing, November 2026. Static prompt bytes dominate input and are cache-discounted; batching halves output cost.

Reading: at one user everything fits the free plan — the product costs **literally zero**. At 10,000 users the cloud infrastructure is still pocket change; the **language model is the only real cost driver**, and the engineering answer is prompt caching, batching, a lite-class primary model, and per-user quotas — exactly why the model is a one-line config value, not a hard-coded name.

10 · Where We Got Stuck (and How We Unblocked)

- **Model retirement mid-project.** The bot answered every message with the fallback apology; logs showed `404 – gemini-2.5-flash is no longer available to new users` across all 10 keys. Fix: switched primary model, then built the benchmark suite so models are swappable config and the chain degrades gracefully.
- **Silent dead replies (waitUntil).** Google's 503s under load respond slowly; retries inside `ctx.waitUntil` exceeded its ~30 s lifetime and Cloudflare cancelled the task mid-flight — no reply, no fallback, nothing in the DB. Fix: inline processing with a 14 s deadline and 8 s per-call timeouts.
- **workers.dev subdomain prerequisite.** First deploy failed until an account subdomain existed (API errors 10031 "unavailable", 10036 "already associated") — the exact pitfall the project spec warns about.
- **New Gemini key format.** AI Studio now issues keys shaped like OAuth codes (`AQ.Ab8RN6…`); they were verified empirically against the models endpoint rather than assumed.
- **Jalali leap-year port bug.** Porting jalaali-js with `Math.floor` instead of truncating division shifted every date by one day (a negative `mod` in the leap calculation). Unit tests with Nowruz and leap-day fixtures caught it.
- **Persian text matching.** Arabic ي/ى vs Persian ی, ك vs ک, zero-width non-joiner and Persian digits all broke fuzzy search until a normalizer folded them.

11 · Operations & Deliverables

- repo — Git repository — code + migrations, **no secrets whatsoever** (all keys live in the Workers secret store)
- live — https://hermes-mini.mojtaba-dev.workers.dev/health
- migrations — `0001_init.sql`, `0002_multiuser.sql` — schema kept separate from code (R7)
- ops — `wrangler deploy`, secrets via `wrangler secret put`, observability on, live logs with `wrangler tail`
- quality — 46 / 46 tests passing; 38-case decision-contract suite; live 4-model benchmark harness
- bonus — Daily morning reminder per user via Cron Triggers (+0.25)